

SPRING MODULITH - COMPLETE REFERENCE GUIDE

A Comprehensive Guide to Modular Monolith Architecture

TABLE OF CONTENTS

1. Core Annotations
2. Event Communication
3. Module Visibility Rules
4. Cross-Module Data Access
5. Async Configuration
6. Named Interfaces for Nested Packages
7. Testing Modularity
8. Best Practices Summary

CORE ANNOTATIONS

1. @ApplicationModule

Purpose: Marks a folder/package as a Spring Modulith module

```
// 📁 product/package-info.java
@ApplicationModule("product")
package com.leoulgetnet.springmodulith.productmodule.product;

import org.springframework.modulith.ApplicationModule;
```

Key Points:

- Defines the module boundary
- All classes under this package belong to the module
- Can specify dependencies: `@ApplicationModule(value = "order", allowedDependencies = {"product"})`

2. ApplicationEventPublisher

Purpose: Publishes events to other modules

```
import org.springframework.context.ApplicationEventPublisher;

@Service
class ProductService {
    private final ApplicationEventPublisher applicationEventPublisher;

    public Product createProduct(Product product) {
        Product saved = repository.save(product);

        // ✅ Publish event
        applicationEventPublisher.publishEvent(
            new ProductCreatedEvent(saved.getId(), saved.getName())
        );

        return saved;
    }
}
```

Key Points:

- Standard Spring event publisher
- Requires `@Transactional` when used with database operations
- Use with `@ApplicationModulePublisher` for cross-module exposure

3. @ApplicationModuleListener

Purpose: Listens to events from other modules

```
import org.springframework.modulith.ApplicationModuleListener;

@Service
class OrderService {

    // ✅ Listens to ProductCreatedEvent from Product module
    @ApplicationModuleListener
    void handleProductCreated(ProductCreatedEvent event) {
        System.out.println("Product created: " + event.id());
        // React to the event
    }
}
```

Key Points:

- Automatically listens to published events
- Executes asynchronously when `@EnableAsync` is used
- One event = one listener (specific handling)
- All listeners for the same event will be triggered

4. @ApplicationModulePublisher

Purpose: Marks methods that expose public API to other modules

```

import org.springframework.modulith.ApplicationModulePublisher;

@Service
class ProductService {

    // ✅ PUBLIC API - other modules can call this
    @ApplicationModulePublisher
    public ProductInfo getProductInfo(Long id) {
        return repository.findById(id)
            .map(p -> new ProductInfo(p.getId(), p.getName(), p.getPrice()))
            .orElseThrow();
    }

    // ✅ PUBLIC API - other modules can call this
    @ApplicationModulePublisher
    public Product createProduct(Product product) {
        Product saved = repository.save(product);
        applicationEventPublisher.publishEvent(
            new ProductCreatedEvent(saved.getId(), saved.getName())
        );
        return saved;
    }

    // ❌ INTERNAL - no annotation, not accessible to other modules
    void internalMethod() {
        // Only used within this module
    }
}

```

Key Points:

- Methods with this annotation can be imported by other modules
- The modularity test WILL PASS when these methods are used
- Methods without this annotation will cause test FAILURE if imported

EVENT COMMUNICATION

Event Publishing Rules

Rule 1: One Event = One Class

✓ **CORRECT - Separate classes for different events**

```
public record ProductCreatedEvent(Long id, String name) {}  
public record ProductUpdatedEvent(Long id, String name, Double price) {}  
public record ProductDeletedEvent(Long id) {}
```

✗ **WRONG - One class for multiple purposes**

```
public record ProductEvent(String type, Long id, String name) {} // Bad practice
```

Rule 2: Events Must Be Public

✓ **CORRECT**

```
public record ProductCreatedEvent(Long id, String name, Double price) {}
```

✗ **WRONG - Not accessible across modules**

```
record ProductCreatedEvent(Long id, String name) {}
```

Rule 3: Events Must Be in Same Package as Module Root or in Named Interface

```
// ✓ Option 1: In root package  
product/  
├─ package-info.java  
├─ ProductCreatedEvent.java // package com...product  
  
// ✓ Option 2: In nested package with @NamedInterface  
product/  
├─ package-info.java  
├─ events/  
│   ├─ package-info.java // @NamedInterface("events")  
│   └─ ProductCreatedEvent.java // package com...product.events
```

Event Listener Rules

Rule 1: All Listeners Get the Event

```
// Product module publishes ONE event
applicationEventPublisher.publishEvent(new ProductCreatedEvent(1L, "Laptop"));

// Order module listener ✔ Gets triggered
@ApplicationModuleListener
void handleProductCreated(ProductCreatedEvent event) { ... }

// Inventory module listener ✔ Gets triggered
@ApplicationModuleListener
void handleProductCreated(ProductCreatedEvent event) { ... }

// Notification module listener ✔ Gets triggered
@ApplicationModuleListener
void handleProductCreated(ProductCreatedEvent event) { ... }
```

Key Points:

- ALL listeners for the event will be triggered
- Unlike Kafka, you cannot filter by specific IDs
- Each listener gets the FULL event object

Rule 2: One Class Per Listener Strategy

✔ CORRECT - Separate classes for different purposes

```
@Service
class OrderService {
    @ApplicationModuleListener
    void handleProductCreated(ProductCreatedEvent event) { ... }
}

@Service
class InventoryService {
    @ApplicationModuleListener
    void handleProductCreated(ProductCreatedEvent event) { ... }
}
```



MODULE VISIBILITY RULES

Visibility Hierarchy

```
// 📁 product/domain/Product.java
package com.leoulgetnet.springmodulith.productmodule.product.domain;

// ✅ CORRECT - Package-private (no 'public')
@Entity
class Product {
    private Long id;
    private String name;
    private Double price;
}
```

Summary Table

Component	Visibility	Reason
Domain Entities	Package-private	Internal to module
Services	Package-private	Internal to module
Repositories	Package-private	Internal to module
Controllers	Package-private	Internal to module
Events	PUBLIC	Cross-module communication
DTOs	PUBLIC	Data transfer between modules

CROSS-MODULE DATA ACCESS

The Solution: @ApplicationModulePublisher

```
// 📁 product/application/ProductService.java
@Service
class ProductService {
    private final ProductRepository repository;
    private final ApplicationEventPublisher publisher;

    @ApplicationModulePublisher
    public ProductInfo getProductInfo(Long id) {
        Product product = repository.findById(id).orElseThrow();
        return new ProductInfo(product.getId(), product.getName(), product.getPrice(), p
    }
}
```

ASYNC CONFIGURATION

```
@Modulith
@EnableAsync // ✅ Enables async event processing
@SpringBootApplication
public class EcommerceApplication { ... }
```

TESTING MODULARITY

```
@SpringBootTest
public class ModularityTests {
    @Test
    void verifyModularStructure() {
        var modules = ApplicationModules.of(EcommerceApplication.class);
        modules.verify(); // ✅ Passes if module boundaries respected
    }
}
```


Modularity Test Matrix

Scenario	Test Result
Importing domain entity from another module	FAILS
Importing repository from another module	FAILS
Importing event from another module	PASSES
Importing service with @ApplicationModulePublisher	PASSES



BEST PRACTICES SUMMARY

- **Module Structure:** Use package-private visibility for internal classes. Use public only for events and DTOs.
- **Events:** One event class = one purpose. Always make events public. Use `@Transactional`.
- **Cross-Module:** Use events for actions/commands, and `@ApplicationModulePublisher` for queries.
- **Testing:** Always run modularity tests and fix violations immediately.